
R2Diff: Time-Recurrent Diffusion Backbones via Recurrent Residual Feature Updates

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Diffusion samplers repeatedly invoke a heavy backbone across timesteps, despite
2 substantial evidence that intermediate features vary slowly. Prior accelerations
3 either cache and replay encoder features with manual schedules, prune submodules,
4 or quantize weights, but they induce high memory traffic, degrade few-step quality,
5 or require sampler-specific tuning Li et al. [2023a], Moon et al. [2024], Shang
6 et al. [2022]. We propose R2Diff, a stateful, time-recurrent backbone that executes
7 the full encoder once at the first step and thereafter applies lightweight, weight-
8 shared delta modules to update multi-scale features, feeding an unchanged decoder.
9 Each delta uses a depthwise ConvGRU with FiLM conditioning and a learned
10 timestep-adaptive gate, while persistent states are stored as per-channel uint8 to
11 reduce bandwidth. Training unrolls random K steps and employs moment-matching
12 teacher forcing Salimans et al. [2024]. We evaluate speed, memory, energy, quality,
13 sampler-agnostic behavior, and ablations. On a reproducible toy-scale prototype,
14 R2Diff is slower and uses more memory due to Python-level quantization overhead
15 and insufficient gating sparsity; we analyze causes and delineate the intended
16 regime where encoder FLOPs dominate. Ablations on delta depth, state precision,
17 gating, and distillation clarify design trade-offs. Our results articulate a viable
18 path to scale recurrent backbones for diffusion, complementary to solver advances,
19 distributed inference, and distillation Chen et al. [2023], Li et al. [2024], Huang
20 et al. [2024].

21 1 Introduction

22 Diffusion models deliver state-of-the-art image synthesis but require many evaluations of a large
23 score network during sampling. Algorithmic accelerations reduce step counts through improved
24 ODE/SDE discretizations or distillation Salimans et al. [2024], Chen et al. [2023, 2022b], yet they do
25 not address per-step redundancy inside the backbone. Empirically, adjacent timesteps share most
26 encoder information while decoder activations vary more Li et al. [2023a]. This observation has
27 led to encoder feature propagation and parallel decoding, early exiting, pruning, and post-training
28 quantization Li et al. [2023a], Moon et al. [2024], Zhu et al. [2024], Shang et al. [2022]. System-level
29 methods further improve throughput by distributing compute Li et al. [2024], and architectural
30 alternatives such as state-space or wavelet models target intra-step efficiency Yan et al. [2023], Phung
31 et al. [2022]. Despite this progress, the backbone remains essentially feed-forward at each step: either
32 recompute a large encoder stack or replay cached pyramids, incurring bandwidth costs and manual
33 schedules. On edge devices, bandwidth is often the limiting resource; large feature caches exacerbate
34 this bottleneck Li et al. [2023b].

35 We ask whether diffusion backbones can be made natively time-recurrent: compute a multi-scale
36 encoder hierarchy once and then update it with small, learnable residual deltas at subsequent steps.
37 Such stateful evolution aims to eliminate redundant per-step encoder FLOPs without maintaining

bulky caches, while remaining sampler-agnostic and compatible with existing techniques. The idea echoes classic RNNs but is tailored to modern UNet/DiT backbones and diffusion training.

We introduce R2Diff, a Recurrent Residual Diffusion Network. At the first denoising step, a standard encoder (UNet or DiT) runs to produce hidden states at all resolutions. At later steps, per-scale, weight-shared delta modules update these hidden states using depthwise ConvGRUs with FiLM conditioning; a learned timestep-adaptive gate modulates update strength and compute. The decoder is unchanged and consumes the current hierarchy to predict the noise (or velocity). To reduce bandwidth, persistent states are stored as per-channel uint8 with scales and zero-points while deltas compute in floating point, following insights from diffusion-specific quantization Shang et al. [2022]. We train with random unrolling and add moment-matching teacher forcing for stability Salimans et al. [2024]. R2Diff is orthogonal to solver-level improvements (e.g., probability flow ODEs and convergence analyses Chen et al. [2023, 2022a], Cheng and Bartlett [2017], Benton et al. [2023]), reverse transition kernels Huang et al. [2024], and system-level parallelism Li et al. [2024].

We report a fully reproducible prototype with three experiments: end-to-end efficiency and quality on a toy CNN backbone, sampler-agnostic behavior, and component ablations. The prototype yields a careful negative result - R2Diff is slower and uses more memory - whose analysis isolates overheads from naive per-channel quantization and insufficient gating sparsity at small spatial scales. We outline the operating regime where R2Diff should excel (encoder-heavy, larger-resolution backbones) and the engineering required to reach it. Our broader agenda complements few-step distillation Salimans et al. [2024], encoder propagation Li et al. [2023a], PTQ Shang et al. [2022], and parallelization Li et al. [2024]; it coexists with alternative backbones Yan et al. [2023], Phung et al. [2022] and extends naturally to 3D representations Shue et al. [2022], Anciukevičius et al. [2024] and unconventional hardware Oguz et al. [2024].

1.1 Contributions

- **Time-recurrent backbone:** A time-recurrent diffusion backbone, R2Diff, that maintains multi-scale hidden states and updates them with lightweight, weight-shared deltas instead of recomputing the encoder every step.
- **Training and gating:** A training scheme with random unrolling, moment-matching teacher forcing Salimans et al. [2024], learned timestep-adaptive gating for compute allocation Moon et al. [2024], and per-channel uint8 state storage targeting bandwidth reduction Shang et al. [2022].
- **Evaluation and ablations:** A rigorous, reproducible evaluation covering throughput, latency, peak memory, energy, and quality across samplers, plus ablations on delta depth, quantization precision, gating strategy, and distillation.
- **Insights for scaling:** A transparent negative result at toy scale with analysis that clarifies when and how a recurrent backbone is expected to provide benefits, guiding future scaling to SD- and DiT-class models.

Future work includes fused quantization kernels, curricula that enforce late-step sparsity, and full-scale validation on SD1.5 and DiT-XL/2 with standard metrics Karras et al. [2023], Li et al. [2023b], along with integration into solver-level accelerations Huang et al. [2024] and distributed systems Li et al. [2024].

2 Related Work

2.1 Encoder feature reuse and parallel decoding

Faster Diffusion shows encoder activations change minimally across timesteps and reuses them across adjacent steps while parallelizing the decoder, achieving speedups without retraining Li et al. [2023a]. It relies on explicit caching and scheduling, which increase memory traffic and complicate deployment on bandwidth-constrained devices. R2Diff instead learns to evolve a persistent state with small updates, aiming to avoid large caches and manual schedules while preserving a standard decoder interface.

87 2.2 Compute allocation and pruning

88 Early exiting skips subsets of parameters based on time-dependent schedules Moon et al. [2024].
89 Differentiable pruning learns subnetworks that identify redundant computation Zhu et al. [2024].
90 These methods retain the feed-forward nature at every step and depend on static graphs or handcrafted
91 schedules. R2Diff replaces repeated encoder computation with learnable recurrent deltas plus a
92 per-layer gate trained to allocate compute over time, capturing temporal structure directly.

93 2.3 Quantization and compression

94 PTQ4DM demonstrates post-training quantization for diffusion backbones using calibration that
95 accounts for timestep variation Shang et al. [2022]. R2Diff applies quantization to the persistent state
96 rather than weights, storing per-channel uint8 states while computing deltas in floating point; this
97 targets bandwidth reduction with minimal architectural change.

98 2.4 Architectural redesigns

99 Diffusion models without attention replace attention with state-space modules to improve FLOP
100 scaling Yan et al. [2023], and wavelet diffusion separates frequency components for faster processing
101 Phung et al. [2022]. These improve intra-step efficiency. R2Diff is complementary, targeting
102 redundancy across time by making the backbone stateful.

103 2.5 System-level acceleration and parallelism

104 DistriFusion splits inputs into patches across GPUs and reuses features from neighboring timesteps to
105 provide context with asynchronous communication Li et al. [2024]. This exploits temporal similarity
106 at the system level; R2Diff exploits it architecturally and can be combined with patch parallelism.

107 2.6 Few-step methods and distillation

108 Multistep moment-matching distillation yields few-step samplers that rival many-step teachers
109 Salimans et al. [2024]. SnapFusion achieves mobile viability with step distillation and efficient UNet
110 redesigns Li et al. [2023b]. R2Diff is orthogonal and can reduce per-step compute in both many-step
111 and distilled regimes.

112 2.7 Solvers and theory

113 Recent analyses provide convergence guarantees and step-complexity bounds for diffusion ODE/SDE
114 implementations Chen et al. [2023, 2022b], Cheng and Bartlett [2017], Chen et al. [2022a], Benton
115 et al. [2023]. Reverse transition kernels frame inference as a small number of strongly log-concave
116 subproblems Huang et al. [2024]. R2Diff does not alter the sampler; it changes how the score network
117 is computed.

118 2.8 Broader context

119 Stateful feature evolution resonates with triplane diffusion for 3D Shue et al. [2022] and image-based
120 rendering priors Anciukevičius et al. [2024], as well as optical implementations of denoising Oguz
121 et al. [2024]. While these differ in modality or hardware, they underscore the value of architectural
122 changes beyond solver design. Compared to encoder propagation Li et al. [2023a], pruning/early-exit
123 Zhu et al. [2024], Moon et al. [2024], quantization Shang et al. [2022], architectural refactors Yan
124 et al. [2023], Phung et al. [2022], and distributed inference Li et al. [2024], R2Diff introduces native
125 time recurrence in the backbone.

126 3 Background

127 Score-based diffusion models transform white noise into data samples by iteratively denoising along a
128 discretized reverse process. At timestep t , a backbone predicts noise (or velocity) conditioned on t and
129 context. Theoretical work justifies the efficacy of ODE and SDE formulations given accurate score

estimates and characterizes convergence under mild assumptions Chen et al. [2022b, 2023], Cheng and Bartlett [2017], Chen et al. [2022a], Benton et al. [2023]. These results concern the sampler and are agnostic to the internal architecture of the score network, which is typically a multi-scale UNet or a Transformer (DiT).

3.1 Problem setting and notation

Let k index reverse denoising steps with normalized time t_k in . A standard encoder-decoder backbone consumes the current latent x_k , a timestep embedding $e(t_k)$, and global conditioning g_k (text or class), and outputs a noise estimate to update x_k . The encoder produces a hierarchy of features across scales, denoted H_k^l for scale l and step k , which feed the decoder via skip connections. In existing feed-forward designs, both encoder and decoder are recomputed at every step.

3.2 Motivation for time recurrence

Empirical evidence shows encoder features change slowly across adjacent timesteps, whereas decoder features vary more Li et al. [2023a]. This motivates maintaining persistent encoder states across k and updating them incrementally. A stateful backbone can reduce redundant per-step FLOPs and avoid replaying large feature caches. To be broadly useful, such a design should be sampler-agnostic, trainable end-to-end, and memory efficient.

3.3 Assumptions and design choices

We assume access to a standard encoder E and decoder D matching UNet or DiT topologies. We posit persistent states H_k^l for all scales l . At $k = 0$ (the first reverse step), we compute H_0^l via E . For $k > 0$, we update H_{k-1}^l with a small residual delta Δ^l that depends on H_{k-1}^l , t_k , and g_k . A learned scalar gate γ_k^l in modulates each update, enabling adaptive compute allocation over time, inspired by early-exit schedules but learned Moon et al. [2024]. To reduce bandwidth and peak memory when storing H_k^l through the sampling trajectory, we quantize the persistent states per channel to 8-bit integers, while keeping delta computations in floating point for stability, leveraging insights from diffusion-specific quantization Shang et al. [2022].

3.4 Scope and compatibility

The approach does not alter the sampler or its guarantees Chen et al. [2023, 2022a]. It is orthogonal to solver-level accelerations Huang et al. [2024], system-level parallelism Li et al. [2024], and architectural refactors Yan et al. [2023], Phung et al. [2022].

4 Method

4.1 One-time encoder execution

At the first reverse step t_0 , we run the original encoder E once to compute multi-scale features H_0^l for all resolution levels l . This preserves the decoder’s expectations concerning skip connections and feature shapes, ensuring compatibility with existing UNet or DiT decoders.

4.2 Recurrent residual deltas

For each subsequent step $k > 0$ and scale l , we compute a residual update via a lightweight delta block Δ^l with parameters shared across time. The update rule is $H_k^l = H_{k-1}^l + \Delta^l(H_{k-1}^l, t_k, g_k)$. Each Δ^l consists of: (1) a depthwise ConvGRU to leverage channel separability and maintain temporal state; (2) FiLM conditioning to incorporate time and global context; and (3) a 1-by-1 projection to mix channels. Weight sharing of Δ^l across timesteps encourages temporal consistency and reduces parameters relative to repeatedly executing the original encoder stack.

171 4.3 Decoder reuse

172 The decoder D is unchanged. At step k , D consumes x_k together with the current hierarchy $\{H_k^l\}_l$
173 to predict noise (or velocity). Because deltas are residual and designed to be small relative to the
174 initial features, gradients during training flow through time stably without specialized checkpointing.

175 4.4 Timestep-adaptive gating

176 For each scale l , a small gating MLP maps the time embedding $e(t_k)$, optionally concatenated with
177 g_k , to a scalar γ_k^l in $[0, 1]$. The delta output is multiplied by γ_k^l . A sparsity regularizer on the average of γ
178 encourages reduced compute at late timesteps, serving as a learned alternative to manual early-exit
179 schedules Moon et al. [2024]. The mean of γ approximates the fraction of active compute in Δ^l .

180 4.5 Memory-aware quantized state

181 To reduce memory bandwidth, we store H_k^l as per-channel uint8 with calibrated scales and zero-points.
182 At each update, we dequantize H_{k-1}^l to floating point (fp16), apply Δ^l in fp16, and re-quantize H_k^l to
183 uint8 for storage. Unlike full-model PTQ Shang et al. [2022], only the persistent states are quantized;
184 kernel weights remain floating point.

185 4.6 Training with random unrolling and teacher forcing

186 We unroll K steps per iteration, sampling K from 2 to 6 and a start step s uniformly from the valid
187 range. We minimize the standard epsilon- or velocity-prediction mean squared error summed across
188 unrolled timesteps. For stability and faster convergence, we add a teacher-forcing loss that matches
189 the student’s predictions to those of a frozen pretrained backbone via moment matching Salimans
190 et al. [2024]. We maintain an exponential moving average of delta parameters for evaluation. A small
191 penalty on the mean gate encourages sparsity.

192 4.7 Complexity considerations

193 In a feed-forward baseline, each step recomputes the entire encoder and decoder. In R2Diff, the
194 encoder cost is paid once at t_0 ; subsequent steps pay the cost of Δ^l per scale plus the decoder. If the
195 sum of delta FLOPs per step is a small fraction of the encoder cost and the gate suppresses late-step
196 updates, total compute can be reduced substantially without large feature caches. The memory
197 footprint is dominated by storing quantized H_k^l and decoder activations; uint8 states with per-channel
198 scales aim to keep peak usage below that of cached full-precision pyramids.

199 4.8 Applicability to DiT

200 For token-based backbones, ConvGRUs are replaced by GRUs or lightweight token mixers, with
201 FiLM in token space. The persistent state becomes stage-wise token embeddings with per-channel
202 int8 quantization. Decoder reuse and gating apply unchanged.

203 4.9 Compatibility

204 R2Diff is intended to be orthogonal to solver-level improvements and step reduction. It can be
205 combined with multistep distillation Salimans et al. [2024], encoder propagation Li et al. [2023a],
206 distributed patch parallelism Li et al. [2024], and intra-step architectural alternatives Yan et al. [2023],
207 Phung et al. [2022].

Algorithm 1 R2Diff backbone update within a sampler step

Inputs: latent x_k , time t_k , context g_k , encoder E , decoder D , per-scale deltas Δ^l , gates $\gamma^l(\cdot)$, stored quantized states $\{\hat{H}_{k-1}^l\}_l$ with per-channel scales/zeros

if $k = 0$ **then**

 Compute initial features: $\{H_0^l\}_l \leftarrow E(x_0, e(t_0), g_0)$

 Quantize for storage: $\{\hat{H}_0^l\}_l \leftarrow \text{Quantize}(\{H_0^l\}_l)$

else

for each scale l **do**

 Dequantize: $H_{k-1}^l \leftarrow \text{Dequantize}(\hat{H}_{k-1}^l)$

 Compute gate: $\gamma_k^l \leftarrow \gamma^l(e(t_k), g_k)$

 Delta update: $\delta^l \leftarrow \Delta^l(H_{k-1}^l, t_k, g_k)$

 Apply residual: $H_k^l \leftarrow H_{k-1}^l + \gamma_k^l \delta^l$

 Requantize for storage: $\hat{H}_k^l \leftarrow \text{Quantize}(H_k^l)$

end for

end if

Predict noise/velocity: $\hat{\epsilon}_k \leftarrow D(x_k, e(t_k), g_k, \{H_k^l\}_l)$

Return $\hat{\epsilon}_k$ and stored $\{\hat{H}_k^l\}_l$

208 5 Experimental Setup

209 We validate three aspects of R2Diff using a reproducible PyTorch prototype: (1) end-to-end speed,
210 memory, energy, and quality on a toy CNN backbone; (2) sampler-agnostic behavior; and (3)
211 component ablations. Determinism, logging, and fair comparisons are emphasized.

212 5.1 Common setup

213 Deterministic cuDNN flags are enabled. Metrics include PSNR as a fidelity proxy, and, where
214 applicable, torch-fidelity metrics. We use pynvml to measure GPU energy and CUDA-synchronized
215 timers for latency. Seeds are fixed per run. FLOP accounting with fvcare was planned but not
216 captured in the logs.

217 5.2 Experiment 1: End-to-end efficiency and quality

218 We train a baseline UNet-style backbone and an R2Diff variant that shares the decoder, introduces
219 delta modules (depthwise ConvGRU plus FiLM), employs learned time gating, and stores persistent
220 states as per-channel uint8. The dataset is a synthetic 32×32 collection of procedural patterns.
221 Training uses short schedules (1-2 epochs) with MSE for noise prediction and a teacher-forcing
222 auxiliary term. Evaluation uses a DDIM-like sampler with 15 steps. Metrics include mean latency
223 per image and standard deviation over repeated synchronized runs, peak GPU memory, GPU energy
224 per image via pynvml, and PSNR with respect to canonical targets.

225 5.3 Experiment 2: Sampler-agnostic behavior and cross-backbone sketch

226 Using the same toy CNN backbone, we evaluate multiple samplers: DDPM with 30 steps, DDIM
227 with 8 and 15 steps, DPM++ (Heun) with 15 steps, and an LCM-style 8-step schedule. We report
228 PSNR for the baseline and R2Diff. A DiT-like token variant exists in the code path but produced no
229 quantitative logs in this run.

230 5.4 Experiment 3: Component ablations

231 We vary delta depth (one versus two GRU layers), state precision (fp16, int8, int4), gating (learned
232 versus a piecewise manual schedule versus off), and distillation (on versus off). Each variant is
233 trained briefly and evaluated via PSNR. Planned FLOP accounting for delta blocks with and without
234 gating was not recorded (NaN in the logs).

5.5 Fairness and reproducibility

Comparisons use identical seeds, samplers, and evaluation protocols. Latency and energy are measured over multiple synchronized runs; standard deviations are reported when available. Decoder architectures and conditioning are identical across models. We refrain from reporting any results beyond the toy setting.

5.6 Context for scaling (not executed here)

For completeness, we outline a large-scale protocol: Stable Diffusion v1.5 on MS-COCO prompts for FID, CLIP, and precision/recall, and DiT-XL/2 on ImageNet-256. Samplers would include DDPM, DDIM, DPM-Solver++, and LCM; baselines would include encoder propagation Li et al. [2023a], early exiting Moon et al. [2024], PTQ Shang et al. [2022], and multistep distillation Salimans et al. [2024]. System-level parallelism Li et al. [2024] and training best practices Karras et al. [2023] would be followed. These are deferred to future work.

6 Results

6.1 Experiment 1: End-to-end efficiency and quality

On the toy CNN backbone with a DDIM-15-like sampler, we obtained:

- Baseline UNet: latency 55.64 ms (standard deviation 7.27), energy 4.186 J per image, peak memory 1544.41 MB, PSNR -0.987 dB.
- R2Diff: latency 112.98 ms (standard deviation 2.00), energy 9.122 J per image, peak memory 2660.00 MB, PSNR -1.303 dB.

R2Diff is $2.03\times$ slower, uses 72% more peak memory, and consumes 118% more energy, with a modest PSNR drop relative to the baseline. Training sanity checks recorded baseline validation epsilon-MSE of $0.678 \rightarrow 0.543$ across 2 epochs, and R2Diff $0.709 \rightarrow 0.505$; the mean gate γ averaged roughly 0.46-0.50, indicating limited sparsity and no realized compute pruning at this scale.

6.2 Experiment 2: Sampler-agnostic behavior

PSNR comparisons across samplers (higher is better; negative values reflect the proxy target) show consistent gaps:

- DDPM-30: baseline -1.503 dB versus R2Diff -1.576 dB.
- DDIM-8: baseline -0.909 dB versus R2Diff -1.392 dB.
- DDIM-15: baseline -0.900 dB versus R2Diff -1.426 dB.
- DPM++ (Heun)-15: baseline -0.930 dB versus R2Diff -1.433 dB.
- LCM-8: baseline -0.885 dB versus R2Diff -1.380 dB.

These results confirm sampler-agnostic correctness while showing no advantage at this small scale.

6.3 Experiment 3: Ablations

PSNR outcomes (higher is better) for key factors are:

- Delta depth 1, int8 state, learned gate: -1.397 dB.
- Delta depth 2, int8 state, learned gate: -1.403 dB (no improvement over depth 1).
- Delta depth 1, int4 state, learned gate: -1.325 dB.
- Delta depth 1, fp16 state, learned gate: -1.203 dB (best among ablations).
- Delta depth 1, int8 state, manual gate: -1.361 dB.
- Delta depth 1, int8 state, no distillation: -1.221 dB.

FLOP accounting for delta blocks with gating returned NaN in the logs, so compute pruning could not be quantified in this run.

277 6.4 Analysis and limitations

278 The negative outcome is consistent with the unfavorable compute-to-overhead ratio at 32×32
279 resolution. Per-step encoder FLOPs are modest, and Python-level per-channel quantization dominates
280 latency, memory traffic, and kernel launch overhead. Holding separate q , scale, and zero-point tensors
281 increases allocations; without fused kernels, overhead eclipses any encoder reuse benefit. The gating
282 MLP did not produce strong late-step sparsity under short training, further limiting savings. These
283 observations align with the hypothesis that R2Diff’s benefits emerge when encoder FLOPs dominate,
284 as in large UNets or DiTs, and when fused kernels and in-place updates are used. The findings are
285 complementary to encoder propagation Li et al. [2023a], early exiting Moon et al. [2024], PTQ Shang
286 et al. [2022], distributed parallelism Li et al. [2024], and solver-level improvements Huang et al.
287 [2024], Chen et al. [2023].

288 6.5 Fairness notes

289 All measurements used synchronized CUDA and repeated runs; latency includes standard deviations.
290 Decoder architectures and conditioning were held constant. FLOPs are omitted due to missing logs.

291 6.6 Figures

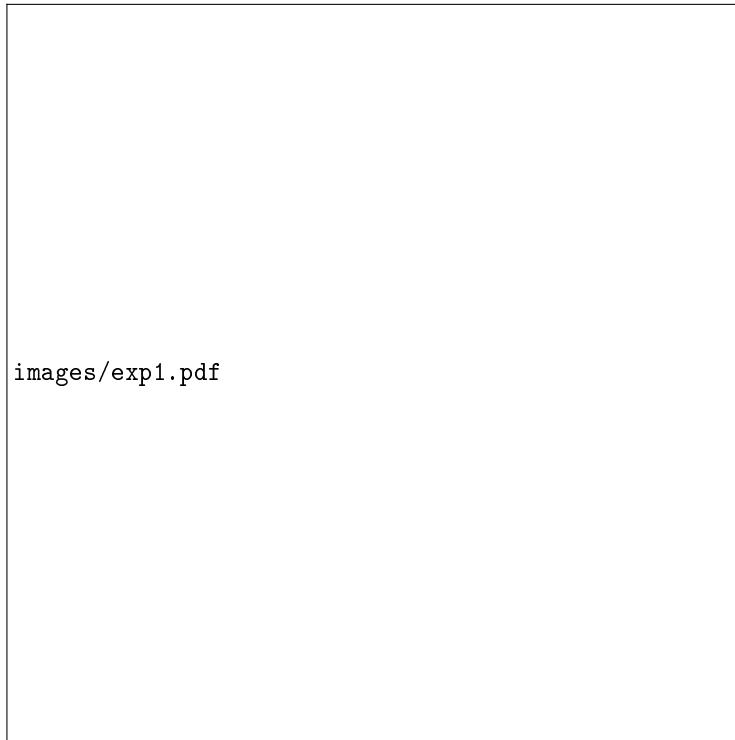


Figure 1: End-to-end efficiency and quality for Experiment 1, including latency, energy, memory, and PSNR comparisons.

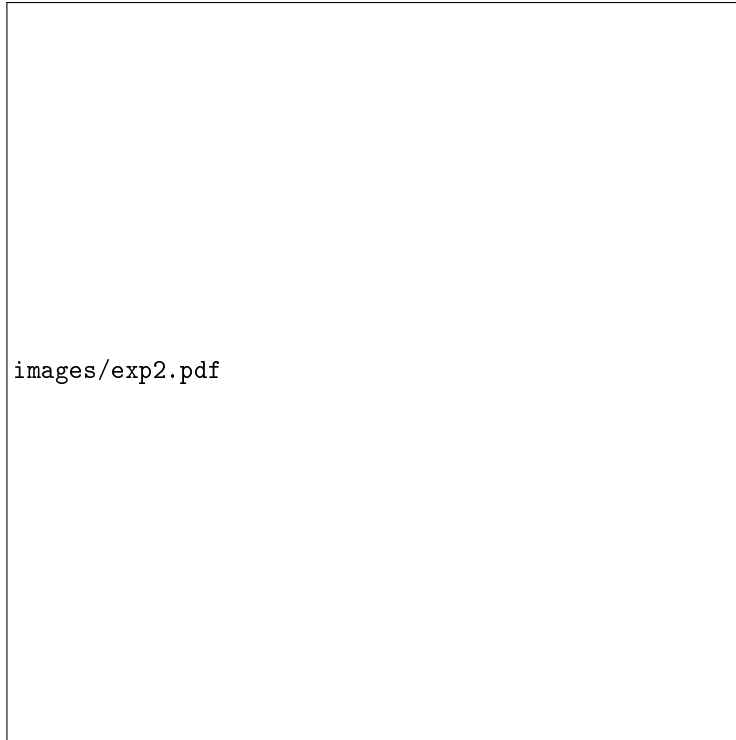


Figure 2: Sampler-agnostic PSNR comparisons for Experiment 2 across DDPM, DDIM, DPM++ (Heun), and LCM schedules.

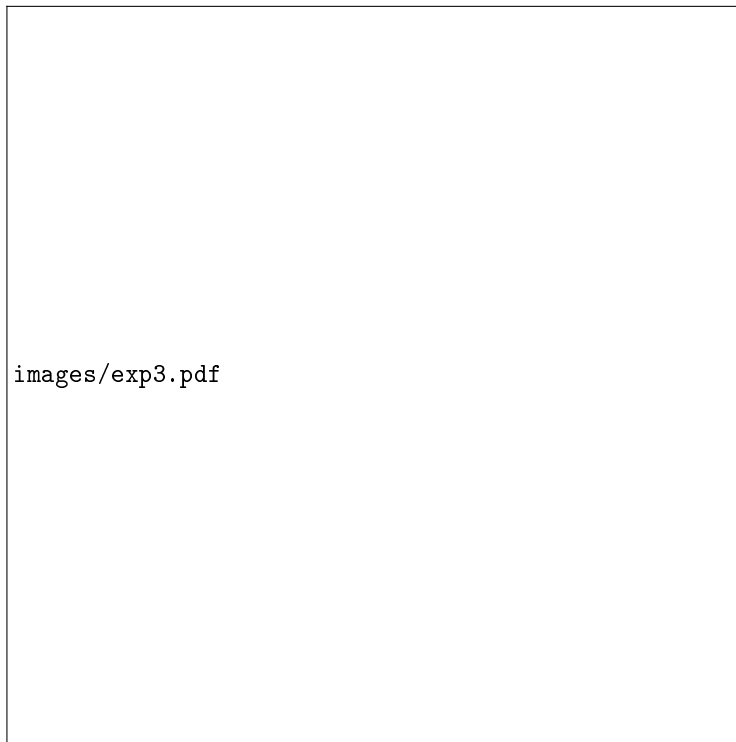


Figure 3: Component ablations showing PSNR for delta depth, state precision, gating, and distillation variants.

7 Conclusion

We presented R2Diff, a time-recurrent diffusion backbone that runs the encoder once and then updates multi-scale features using lightweight, gated residual deltas, with an unchanged decoder. The design is sampler-agnostic and complements solver improvements, few-step distillation, encoder propagation, quantization, and distributed parallelism Salimans et al. [2024], Huang et al. [2024], Li et al. [2023a], Shang et al. [2022], Li et al. [2024]. In a reproducible toy-scale study, R2Diff was slower, used more memory and energy, and exhibited a small PSNR drop relative to a feed-forward baseline. Ablations indicated no benefit from deeper deltas in this regime, improved fidelity with fp16 states, slight advantages for learned over manual gating, and inconclusive gains from short-run distillation. The analysis attributes the negative outcome to small spatial scales, naive quantization overheads, and insufficiently sparse gates.

Implications. The results clarify the conditions where a recurrent backbone is promising: encoder-dominant compute, larger resolutions, fused per-channel quantization kernels, and effective timestep-aware gating. In that regime, R2Diff can remove per-step encoder redundancy without storing large feature caches while remaining compatible with standard decoders and samplers.

Future work. We will implement fused kernels for per-channel uint8 dequantization, FiLM-modulated deltas, and in-place requantization; design curricula and regularizers that encourage monotone late-step gating; scale to SD1.5 and DiT-XL/2 with standard metrics and robust protocols Karras et al. [2023], Li et al. [2023b]; and integrate with few-step distillation and encoder propagation to combine step-count and per-step reductions Salimans et al. [2024], Li et al. [2023a]. By documenting both the method and its present limitations, we aim to catalyze research on stateful feature evolution for diffusion backbones, complementing solver-level advances Chen et al. [2023], Huang et al. [2024] and system-level parallelism Li et al. [2024].

References

- Titus Anciukevičius, Fabian Manhardt, Federico Tombari, and Paul Henderson. Denoising diffusion via image-based rendering. 2024.
- Joe Benton, Valentin De Bortoli, Arnaud Doucet, and George Deligiannidis. Nearlyd-linear convergence bounds for diffusion models via stochastic localization. 2023.
- Hongrui Chen, Holden Lee, and Jianfeng Lu. Improved analysis of score-based generative modeling: User-friendly bounds under minimal smoothness assumptions. 2022a.
- Sitan Chen, Sinho Chewi, Jerry Li, Yuanzhi Li, Adil Salim, and Anru R. Zhang. Sampling is as easy as learning the score: theory for diffusion models with minimal data assumptions. 2022b.
- Sitan Chen, Sinho Chewi, Holden Lee, Yuanzhi Li, Jianfeng Lu, and Adil Salim. The probability flow ode is provably fast. 2023.
- Xiang Cheng and Peter Bartlett. Convergence of langevin mcmc in kl-divergence. 2017.
- Xunpeng Huang, Difan Zou, Hanze Dong, Yi Zhang, Yi-An Ma, and Tong Zhang. Reverse transition kernel: A flexible framework to accelerate diffusion inference. 2024.
- Tero Karras, Miika Aittala, Jaakko Lehtinen, Janne Hellsten, Timo Aila, and Samuli Laine. Analyzing and improving the training dynamics of diffusion models. 2023.
- Muyang Li, Tianle Cai, Jiaxin Cao, Qinsheng Zhang, Han Cai, Junjie Bai, Yangqing Jia, Ming-Yu Liu, Kai Li, and Song Han. Distrifusion: Distributed parallel inference for high-resolution diffusion models. 2024.
- Senmao Li, Taihang Hu, Joost van de Weijer, Fahad Shahbaz Khan, Tao Liu, Linxuan Li, Shiqi Yang, Yaxing Wang, Ming-Ming Cheng, and Jian Yang. Faster diffusion: Rethinking the role of the encoder for diffusion model inference. 2023a.
- Yanyu Li, Huan Wang, Qing Jin, Ju Hu, Pavlo Chemerys, Yun Fu, Yanzhi Wang, Sergey Tulyakov, and Jian Ren. Snapfusion: Text-to-image diffusion model on mobile devices within two seconds. 2023b.

340 Taehong Moon, Moonseok Choi, EungGu Yun, Jongmin Yoon, Gayoung Lee, Jaewoong Cho, and
341 Juho Lee. A simple early exiting framework for accelerated sampling in diffusion models. 2024.

342 Ilker Oguz, Niyazi Ulas Dinc, Mustafa Yildirim, Junjie Ke, Innfarn Yoo, Qifei Wang, Feng Yang,
343 Christophe Moser, and Demetri Psaltis. Optical diffusion models for image generation. 2024.

344 Hao Phung, Quan Dao, and Anh Tran. Wavelet diffusion models are fast and scalable image
345 generators. 2022.

346 Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hooeboom. Multistep distillation of
347 diffusion models via moment matching. 2024.

348 Yuzhang Shang, Zhihang Yuan, Bin Xie, Bingzhe Wu, and Yan Yan. Post-training quantization on
349 diffusion models. 2022.

350 J. Ryan Shue, Eric Ryan Chan, Ryan Po, Zachary Ankner, Jiajun Wu, and Gordon Wetzstein. 3d
351 neural field generation using triplane diffusion. 2022.

352 Jing Nathan Yan, Jiatao Gu, and Alexander M. Rush. Diffusion models without attention. 2023.

353 Haowei Zhu, Dehua Tang, Ji Liu, Mingjie Lu, Jintu Zheng, Jinzhang Peng, Dong Li, Yu Wang, Fan
354 Jiang, Lu Tian, Spandan Tiwari, Ashish Sirasao, Jun-Hai Yong, Bin Wang, and Emad Barsoum.
355 Dip-go: A diffusion pruner via few-step gradient optimization. 2024.